PYTHON ACTIVITY 1

Ten case studies | Python solutions

Algorithms • pseudocode • flow logic • runnable programs

V. Alekhya | Roll No. 26000141 | CSE-2

Variables, data types, operators, conditions, loops and strings

Lists, nested lists, tuples, tuple unpacking and pattern generation

```
Pattern example: print('*' * 5) → *****
```

Programs use sample records and simple loops; update rates and thresholds to match your class policy.

HOW TO USE THESE SOLUTIONS

Each topic has two slides

1 Model: sample policy, pseudocode and flowchart sequence.

2 Program: full Python solution split into two readable columns.

The rules were not specified in the prompt, so every numeric threshold and rate is labeled as a sample assumption.

The code uses core variables, tuples, lists, loops, conditions and operators; it does not rely on external libraries.

CASE 1 • MODEL

Ten case studies

Topics 1–5

1 Scholarship allocation
2 Progressive electricity billing
3 Hospital billing and classification
4 Employee payroll
5 Mobile plan optimization

Topics 6–10

6 Railway fares and reservations
7 Retail billing and discounts
8 Manufacturing profitability
9 Multi-currency travel budget
10 Campus event break-even

CASE 1 • COMPLETE PROGRAM

University Scholarship Allocation

Policy assumptions

Aggregate ≥ 80; each subject ≥ 60; attendance ≥ 75%; family income ≤ ■300,000.
Rank by aggregate, attendance, lower income, then roll number.

Pseudocode

FOR each applicant: check thresholds; add eligible records.
WHILE eligible list is not empty: choose the best by tie-break rules; print and remove.

INPUT → CHECK → FILTER → RANK → REPORT

CASE 1 • COMPLETE PROGRAM

University Scholarship Allocation | Python

```
applicants = [
("Asha",101,88,82,91,84,90,75,180000),
("Ravi",102,84,78,86,82,81,80,220000),
("Meera",103,92,89,94,90,93,88,250000)]
eligible = []
for name, roll, m1, m2, m3, m4, agg, att, income in applicants:
if agg >= 80 and min(m1,m2,m3,m4) >= 60 and att >= 75 and income <=
300000:
eligible.append((agg,att,-income,-roll,name,roll))
rank = 1
while len(eligible) > 0:
best = eligible[0]
for record in eligible:
if record[:4] > best[:4]: best = record
print(rank,best[4],"roll",best[5],"aggregate",best[0])
eligible.remove(best)
rank = rank + 1
```

CASE 5 • MODEL

Progressive Electricity Billing

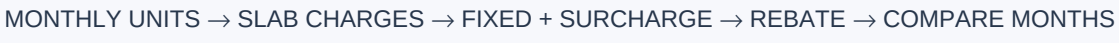
Sample policy / assumptions

Tariff: first 100 units ■2, next 100 ■3, above 200 ■5. Fixed charge ■80; surcharge 5%; rebate ■50 below 100 units. Increase >30% is unusual.

Pseudocode

```
Loop over months. Split units into 3 slabs. Add fixed charge and surcharge. Apply rebate if eligible. Compare each bill with the prior month.
```

Flowchart



CASE 6 • COMPLETE PROGRAM

Progressive Electricity Billing | Python

```
months=[("Jan",80),("Feb",120),("Mar",260)]
bills=[]
for month,units in months:
first=min(units,100)
second=min(max(units-100,0),100)
third=max(units-200,0)
energy=first*2+second*3+third*5
bill=(energy+80)*1.05
if units<100: bill=bill-50
bills.append((month,units,bill))
for month,units,bill in bills:
print(month,units,"units: ■",round(bill,2))
for i in range(1,len(bills)):
old=bills[i-1][2]; new=bills[i][2]
increase=(new-old)*100/old
if increase>30:
print("Unusual:",bills[i][0],round(increase,1),"%")
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 7 • MODEL

Hospital Billing and Patient Classification

Sample policy / assumptions

```
Reject negative charges. Apply 10% concession if bill before discount ≥ ■10,000. Final bill classes: low ≤ ■2,000; medium ≤ ■10,000; high above.
```

Pseudocode

For each patient, add consultation, tests and treatment. If valid, apply threshold concession. Use conditional bands to classify final payment.

Flowchart

ENTER CHARGES → VALIDATE → TOTAL → CONCESSION → CLASSIFY

CASE 8 • COMPLETE PROGRAM

Hospital Billing and Patient Classification | Python

```
patients=[("P101",500,1800,7000),("P102",300,600,900)]
for patient,consult,tests,treatment in patients:
if consult<0 or tests<0 or treatment<0:
print(patient,"invalid")
else:
total=consult+tests+treatment
concession=0
if total>=10000: concession=total*0.10
payable=total-concession
if payable<=2000: group="LOW"
elif payable<=10000: group="MEDIUM"
else: group="HIGH"
print(patient,"total ■",total,"discount ■",concession)
print("Pay ■",round(payable,2),"class",group)
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 9 • MODEL

Employee Payroll and Deduction Analysis

Sample policy / assumptions

Gross = basic + allowances + overtime + incentive. Tax: 0% up to ■25,000; 10% next ■25,000; 20% above. Other deduction = 5% of basic.

Pseudocode

Compute gross pay. Split gross across three tax bands. Subtract tax and the basic-pay deduction. Print salary statement.

Flowchart

PAY INPUTS → GROSS → PROGRESSIVE TAX → OTHER DEDUCTION → NET STATEMENT

CASE 10 • COMPLETE PROGRAM

Employee Payroll and Deduction Analysis | Python

```
staff=[("Anu",30000,6000,2000,1500),("Dev",22000,4000,1200,800)]
for name,basic,allowance,overtime,incentive in staff:
gross=basic+allowance+overtime+incentive
first=min(gross,25000)
second=min(max(gross-25000,0),25000)
third=max(gross-50000,0)
tax=second*0.10+third*0.20
other=basic*0.05
net=gross-tax-other
print("="*35)
print("SALARY STATEMENT:",name)
print("Gross ■",round(gross,2))
print("Tax ■",round(tax,2),"Other ■",round(other,2))
print("NET PAY ■",round(net,2))
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 11 • MODEL

Telecommunications Cost Optimization

Sample policy / assumptions

```
A plan is eligible only if it covers validity and total cost is within budget. Plans store data, minutes, days, base price and excess rates. Choose the cheapest eligible plan.
```

Pseudocode

For each plan, calculate excess data and minutes and total charge. Check validity and budget. Track the lowest eligible price.

Flowchart

CASE 12 • COMPLETE PROGRAM

Telecommunications Cost Optimization | Python

```
data_used=8; minutes_used=300; days_needed=28; budget=500
plans=[("Lite",5,250,28,249,40,0.5),
("Smart",10,500,30,399,30,0.4),
("Max",20,1000,56,599,20,0.3)]
best=""; best_cost=budget+1
for name,data,mins,days,base,dr,mr in plans:
cost=base+max(data_used-data,0)*dr+max(minutes_used-mins,0)*mr
ok=days>=days_needed and cost<=budget
print(name,"■",round(cost,2),"eligible",ok)
if ok and cost<best_cost: best=name; best_cost=cost
if best=="": print("No eligible plan")
else: print("Best plan",best,"■",round(best_cost,2))
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 13 • MODEL

Railway Fare and Reservation Calculator

Sample policy / assumptions

Sleeper ■1.20/km; AC ■2.50/km. Above 500 km, add ■0.20/km to fare. Age 60+ receives 40% concession. Reservation ■40; cancellation penalty is 25% of fare.

Pseudocode

Select class rate. Calculate distance fare and surcharge. Apply age concession. Add reservation fee and calculate refund after penalty.

Flowchart

PASSENGER → CLASS + DISTANCE → AGE CONCESSION → RESERVATION → REFUND

CASE 14 • COMPLETE PROGRAM

Railway Fare and Reservation Calculator | Python

```
people=[("Asha",24,650,"S"),("Ravi",67,420,"A")]
for name,age,km,cls in people:
if cls=="S": rate=1.2
else: rate=2.5
fare=km*rate
if km>500: fare=fare+(km-500)*0.20
if age>=60: fare=fare*0.60
total=fare+40
refund=total-fare*0.25
print(name,cls,"ticket ■",round(total,2))
print("Cancel refund ■",round(refund,2))
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 15 • MODEL

Retail Billing with Multiple Discount Rules

Sample policy / assumptions

Tax: grocery 5%, clothing 12%, electronics 18%. Discount 5% at ■2,000 and 8% at ■5,000, capped at ■1,000. Tax applies after discount.

Pseudocode

Sum item quantities times prices. Select discount band and cap. Apply discount proportionally to each line before category tax. Print invoice.

Flowchart

ITEMS + QTY → SUBTOTAL → THRESHOLD DISCOUNT → CATEGORY TAX → INVOICE

CASE 16 • COMPLETE PROGRAM

Retail Billing with Multiple Discount Rules | Python

```
items=[("Rice","grocery",5,80),("Shirt","clothing",2,900),
("Headphones","electronics",1,1800)]
subtotal=0
for name,cat,qty,price in items: subtotal+=qty*price
if subtotal>=5000: rate=0.08
elif subtotal>=2000: rate=0.05
else: rate=0
discount=min(subtotal*rate,1000); tax_total=0
for name,cat,qty,price in items:
tax_rate=0.05
if cat=="clothing": tax_rate=0.12
elif cat=="electronics": tax_rate=0.18
taxable=qty*price*(1-discount/subtotal)
tax=taxable*tax_rate; tax_total+=tax
print(name,qty,"x ■",price,"tax ■",round(tax,2))
print("Subtotal",subtotal,"Discount",round(discount,2))
print("TOTAL ■",round(subtotal-discount+tax_total,2))
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 17 • MODEL

Manufacturing Cost and Profitability

Sample policy / assumptions

Include material cost, 5% material wastage, labour and machine cost. Profit target is 20% margin on total cost.

Pseudocode

Calculate costs and revenue. Profit = revenue - total cost. Margin = profit / cost × 100. Compare with target.

Flowchart

ORDER → MATERIAL + WASTAGE → LABOUR + MACHINE → PROFIT → TARGET CHECK

CASE 18 • COMPLETE PROGRAM

Manufacturing Cost and Profitability | Python

```
units=500
material=units*2.5*40
wastage=material*0.05
labour=units*18; machine=units*10
total_cost=material+wastage+labour+machine
revenue=units*90
profit=revenue-total_cost
margin=profit*100/total_cost
print("Material ■",material,"Wastage ■",wastage)
print("Labour ■",labour,"Machine ■",machine)
print("Cost ■",total_cost,"Revenue ■",revenue)
print("Profit ■",profit,"Margin",round(margin,2),"%")
if margin>=20: print("Target met")
else: print("Target not met")
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 19 • MODEL

Multi-Currency Travel Budget Analysis

Sample policy / assumptions

Exchange rates are supplied in ■ per unit. Add a 2% conversion fee. Group spending by category, compare total with budget, and report the largest category.

Pseudocode

Unpack each category, amount and currency. Find its rate. Convert to rupees with fee. Accumulate category totals and track the maximum.

Flowchart

EXPENSE TUPLES → RATE LOOKUP → CONVERT + FEE → CATEGORY TOTALS → BUDGET

CASE 20 • COMPLETE PROGRAM

Multi-Currency Travel Budget Analysis | Python

```
rates=(("USD",83.0),("EUR",90.0),("JPY",0.56))
expenses=[("Food",120,"USD"),("Travel",90,"EUR"),
("Hotel",180,"USD"),("Tickets",8000,"JPY")]
budget=35000; totals=[]; grand=0
for category,amount,currency in expenses:
rate=0
for code,value in rates:
if code==currency: rate=value
converted=amount*rate*1.02; grand+=converted
found=False
for i in range(len(totals)):
if totals[i][0]==category:
totals[i]=(category,totals[i][1]+converted); found=True
if not found: totals.append((category,converted))
largest=totals[0]
for item in totals:
if item[1]>largest[1]: largest=item
print("Spend ■",round(grand,2),"Budget ■",budget)
print("Largest:",largest[0],"■",round(largest[1],2))
if grand<=budget: print("Within budget")
else: print("Over budget")
```

Sample inputs are editable. Change policy values to match the assigned rules.

CASE 21 • MODEL

Campus Event Financial Planning

Sample policy / assumptions

Fixed costs: venue, equipment and transport. Certificates and catering are per attendee. Sponsorship reduces fixed costs. Minimum registrations use a fee less both per-person costs

Pseudocode

fixed = venue + equipment + transport
variable/person = certificate + catering
contribution = fee - variable/person
shortfall = max(total cost - fees - sponsor, 0)
minimum = ceiling(max(fixed - sponsor, 0) / contribution)

Flowchart

COST INPUTS → FIXED + VARIABLE → SPONSORSHIP → CONTRIBUTION / PERSON → BREAK EVEN

CASE 22 • COMPLETE PROGRAM

Campus Event Financial Planning | Python

```
venue=18000; equipment=7500; transport=9000
certificate_each=25; catering_each=220
fee=500; sponsor=12000; expected=100
fixed=venue+equipment+transport
variable=certificate_each+catering_each
total=fixed+variable*expected
shortfall=max(total-fee*expected-sponsor,0)
contribution=fee-variable
if contribution<=0:
print("Fee must exceed per-person costs")
else:
recover=max(fixed-sponsor,0)
minimum=(recover+contribution-1)//contribution
print("Estimated cost ■",total)
print("Sponsorship shortfall ■",shortfall)
print("Break-even registrations",minimum)
```

Sample inputs are editable. Change policy values to match the assigned rules.